

LAB 2: Pthreads

Embedded Processing Architectures

Zhang Ruida and Hugo Bueno

April 15th, 2024

Parallelization Implementation

The basic matrix calculation algorithm is for two multiplied matrices A, B performs n multiplication and addition of n elements in row i of matrix A and n elements in column j of matrix B to obtain c_{ij} in the resulting matrix C , and repeats this process $n \times n$ times to obtain elements in the entire matrix C , where n is the size of the matrix. Therefore, the whole process needs to compute $n \times n \times n$ multiplications. **In the basic algorithm, the n^3 multiplications are processed in series**, so when the size of the matrix n is very large, it takes a lot of time.

In order to realize the parallel version of the matrix multiplication algorithm based on **pthread**, the task interval can be divided between multiple threads. For a matrix of size n , when m threads are used for processing, each thread can be responsible for calculating the result of m/n row, that is, **each thread will be responsible for calculating a subset of the result matrix C , as seen in Equation (1)**. This can reduce the running time of the program when the number of threads is appropriate.

$$A * B = C \rightarrow \begin{cases} \text{thread 1} \rightarrow \\ \vdots \\ \text{thread } n \rightarrow \end{cases} \begin{pmatrix} a_{00} & \cdots & a_{0n} \\ \vdots & \ddots & \vdots \\ a_{n0} & \cdots & a_{nn} \end{pmatrix} * \begin{pmatrix} b_{00} & \cdots & b_{0n} \\ \vdots & \ddots & \vdots \\ b_{n0} & \cdots & b_{nn} \end{pmatrix} = \begin{pmatrix} c_{00} & \cdots & c_{0n} \\ \vdots & \ddots & \vdots \\ c_{n0} & \cdots & c_{nn} \end{pmatrix} \quad (1)$$

Our biggest trouble was in the implementation process is the data transfer: for each thread, it needs to access the data of its own calculation part. At first, we considered using global variables for processing, which made it impossible to use malloc for dynamic memory management, **so we ended up using structs for data transfer, and threads can access the address of the structure to operate on the data in the structure.**

It is noteworthy that, in order to avoid the calculation differences caused by different matrices when changing the number of threads in the same matrix size, the matrix should calculate all the different number of threads when generating a matrix at random, thus eliminating interference factors.

Results

Table 1 below shows the results for this algorithm implementation in the ODROID-XU4Q board. The decrease of the execution time with the number of threads can be better appreciated in Figures 1 to 5.

Execution time in ms		Number of threads				
		1	2	4	8	16
Matrix size	64	132	42	19	14	13
	128	216	187	79	47	44
	256	1.456	827	474	328	254
	512	11.779	5.854	3.079	2.838	2.294
	1024	293.626	146.179	76.101	42.418	36.153

Table 1. Execution time results in ms

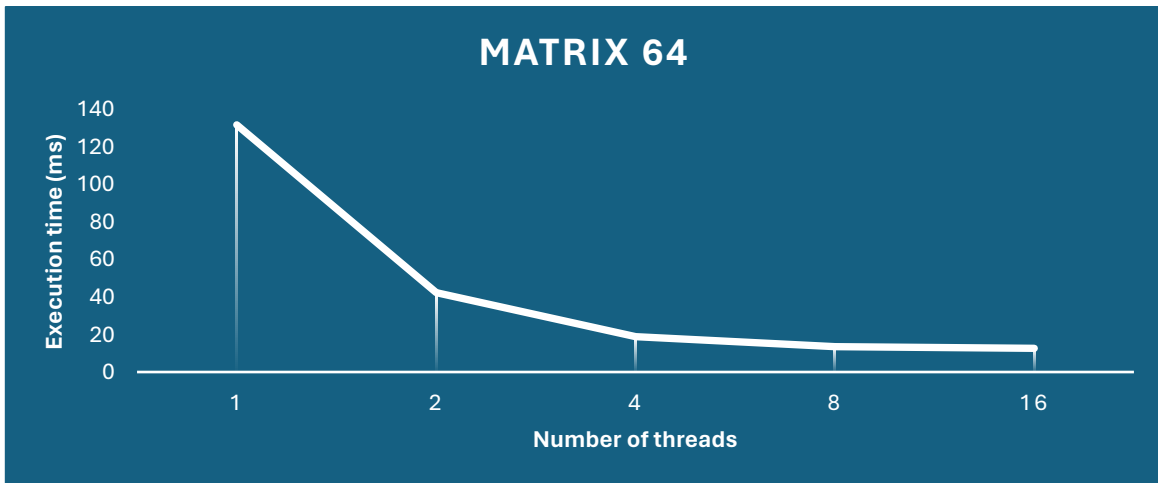


Figure 1. Execution time of the algorithm for different number of threads (64x64 matrix)

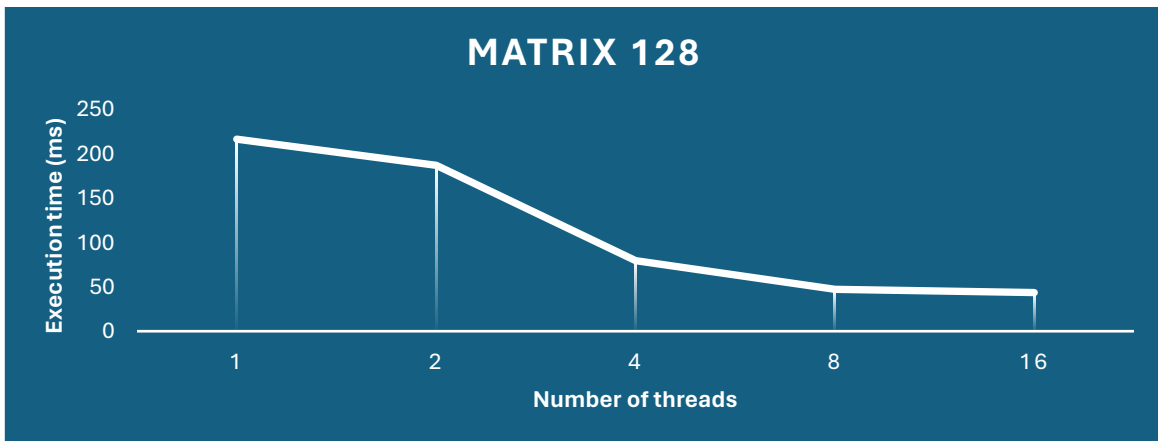


Figure 2. Execution time of the algorithm for different number of threads (128x128 size matrix)

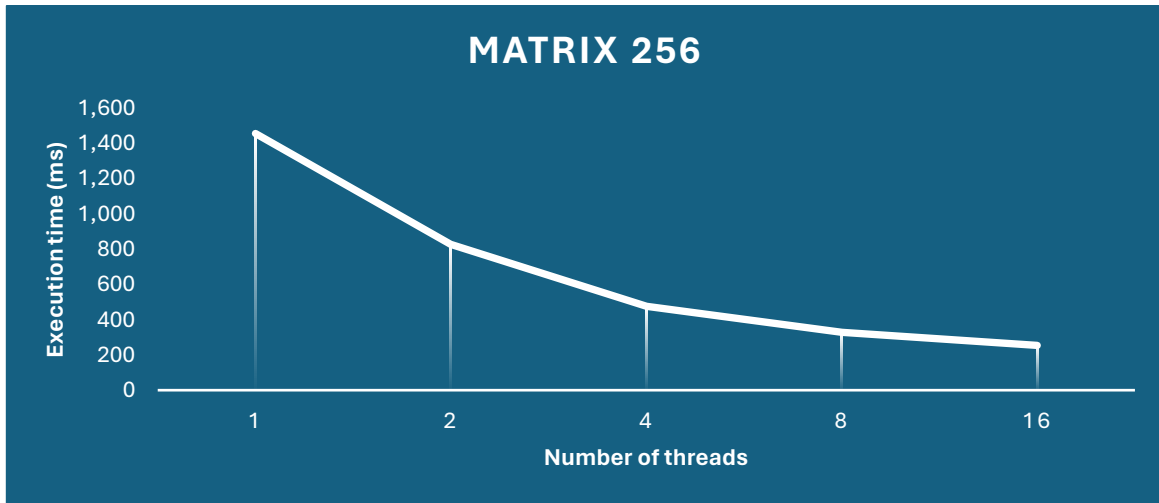


Figure 3. Execution time of the algorithm for different number of threads (256x256 size matrix).

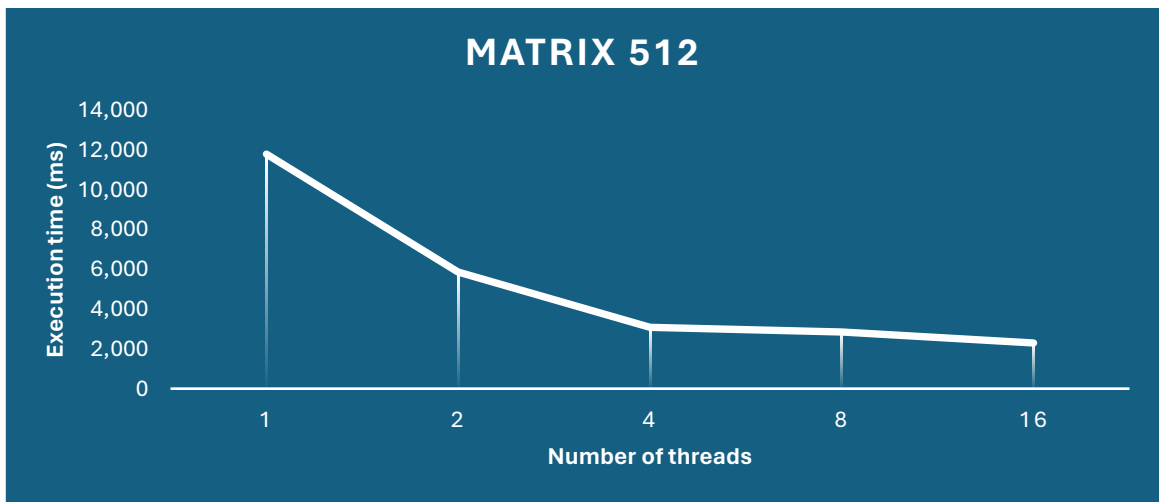


Figure 4. Execution time of the algorithm for different number of threads (512x512 size matrix)

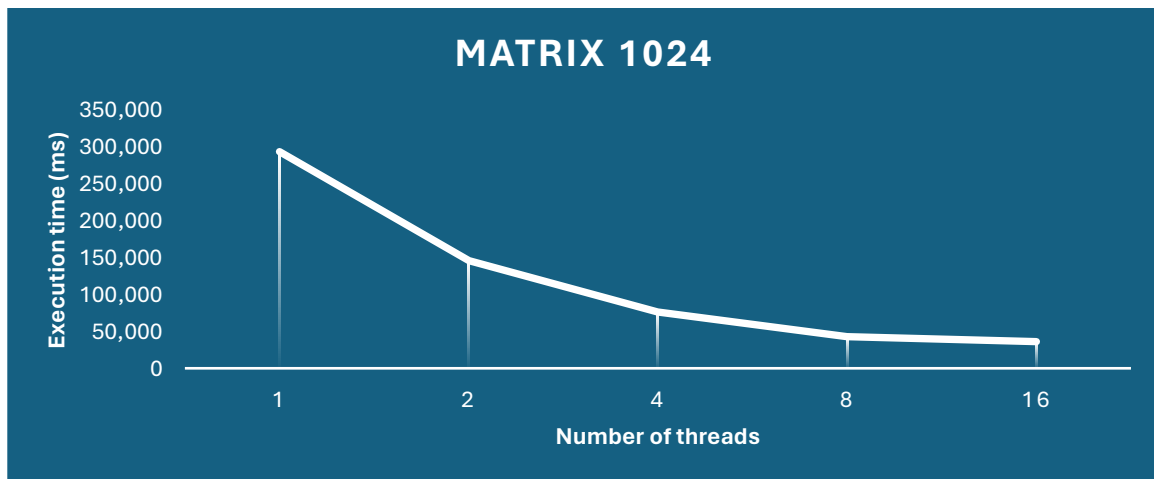


Figure 5. Execution time of the algorithm for different number of threads (104x1024 size matrix)

As expected, the increase on the number of threads reduces the execution time with almost an exponential decrease, due to the fact that the number of processes being realized in parallel is increased by a factor of 2. However, this improvement is limited when passing from 8 to 16 threads. The reason is that, when using a Samsung Exynos5422 System on Chip (SoC) that includes an Arm big.LITTLE multi-core processor (4 big Cortex-A15 cores and 4 LITTLE CortexA7), **only 8 cores may be used in parallel**, thus being a bottleneck for the number of processes which can be carried out in parallel.

This explanation applies for all figures, meaning the same behavior is expected for whatever the size of the multiplied matrixes is. Nevertheless, the absolute value of the considered execution times is clearly higher, as seen in Figure 6. The execution times for matrixes of 1024 elements is on another order of magnitude, not even being able to appreciate the phenomena described before for other sizes.

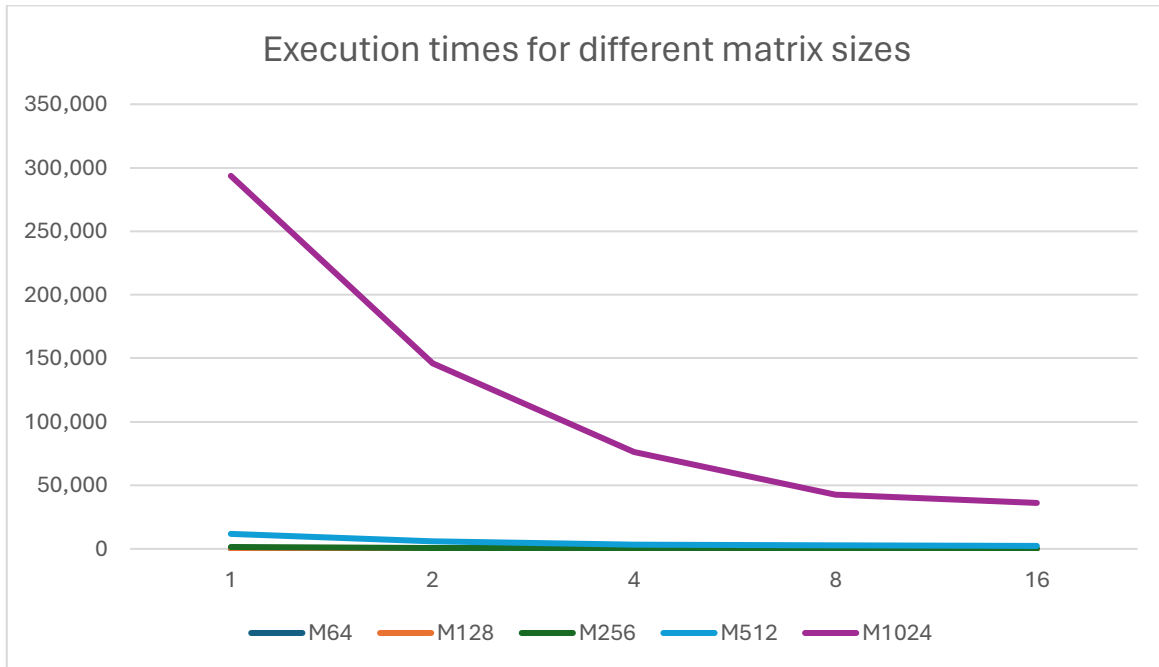


Figure 6. Execution times for different matrix sizes.